

Deployment & SSH Guide — DefectScan Backend

A short reference for how the FastAPI backend is hosted, accessed over SSH, and updated. Use this as a talking script: "I used **X** to host it, **Y** to connect, **Z** to serve it..."

The deployment stack (what to name in the discussion)

Layer	Technology used	Role
Cloud host	AWS EC2 (Amazon Linux, ec2-user)	The virtual server the backend runs on
Remote access	SSH (OpenSSH client) with a .pem key pair	How we log into the server
File transfer / deploy	SCP (scp over SSH)	How updated code files are copied to the server
ASGI server	Uvicorn	Runs the FastAPI app (the Python web process)
Reverse proxy	Nginx	Listens on port 80 and forwards traffic to Uvicorn
Process manager	systemd (service name app)	Keeps the backend running and auto-restarts it
Runtime	Python 3.9 in a virtual environment (venv)	Isolated dependency environment
Logs	journalctl	Reads the service's live logs

One-line summary to say out loud:

"The backend is deployed on an **AWS EC2** instance. I connect to it over **SSH** using a **.pem** key pair, and deploy code by copying files with **SCP**. The app runs as a **systemd** service that launches **Uvicorn**, and **Nginx** sits in front as a reverse proxy on port 80."

1. Connect over SSH

```
# Standard OpenSSH client (built into Windows, macOS, Linux)
ssh -i path/to/key.pem ec2-user@<EC2_PUBLIC_IP>
```

- **User:** **ec2-user**
- **Authentication:** key-based (a **.pem** private key) — no passwords.
- On Windows, if OpenSSH rejects the key's permissions, copy the **.pem** to a folder locked to your user account so its ACLs are tightened before connecting.
- The app lives in **~/app/** on the server (**main.py**, **routes/**, the model weights, **uploads/**).

The EC2 public IP can change if the instance is stopped/started (no Elastic IP attached). Verify the host is live first: `curl -s -o /dev/null -w "%{http_code}" http://<IP>/docs → 200`.

2. Deploy a code change (file update)

```
# 1. Back up the current file on the server first
ssh -i key.pem ec2-user@<IP> "cp ~/app/routes/report_routes.py
~/app/routes/report_routes.py.bak.$(date -u +%Y%m%d-%H%M%S)"

# 2. Copy the new file up with SCP
scp -i key.pem "routes/report_routes.py" ec2-user@<IP>:~/app/routes/

# 3. Restart the service so Uvicorn reloads the new code
ssh -i key.pem ec2-user@<IP> "sudo systemctl restart app"
```

Expect a brief `502` from Nginx right after a restart while the ML model loads, then `200`.

3. Manage the running service (systemd)

```
sudo systemctl status app      # is it running?
sudo systemctl restart app     # restart after a deploy
sudo systemctl stop app        # stop
journalctl -u app -f           # follow live logs (Ctrl-C to exit)
```

4. Add a Python dependency on the server

```
ssh -i key.pem ec2-user@<IP>
cd ~/app
source venv/bin/activate
pip install <package>          # e.g. google-genai for the Gemini chat feature
sudo systemctl restart app
```

5. Environment variables the backend reads

Set these in the systemd service environment (absent values degrade gracefully):

Variable	Purpose	If missing
SECRET_KEY	JWT signing secret	falls back to an insecure default — set in production
GEMINI_API_KEY	Google Gemini key for the AI chat mode	/chat/ai returns 503
GEMINI_MODEL	Gemini model name	defaults to gemini-2.5-flash

Notes / hardening for production (mention as "future work")

- Attach an **Elastic IP** so the public address stops changing.
- Put **HTTPS/TLS** in front of Nginx (e.g. Let's Encrypt) instead of plain HTTP.
- Move SECRET_KEY and all API keys to the service environment — never hard-code them.
- The model weights file (mobilenetv3_defect_4class.pt) is **not** in this repo (too large for GitHub); it is deployed separately to ~/app/ on the EC2 box.